Named Axes as a Systems Discipline: Hybrid Compile-Time and Runtime Coordination for Tensor Programs

[TODO: author block — fill at submission time]
authors@example.com

Abstract

Tensor programs routinely fail on *axis-bug* classes that positional shape systems cannot catch: silent broadcasts across mis-named dimensions, transposed contractions, reductions over the wrong axis. Existing named-axis libraries push the check from "silent wrong output" to "runtime exception" but leave the host language's static type system unused. We present a hybrid design that encodes axis information across the compile-time / runtime boundary, with three contributions: (i) a `_tex` user-defined literal that parses a LaTeX-math subset (e.g. `R"(c_{ij} = a_i b_j)"_tex`) into a named-axis expression AST whose structural indices the type system can reason about; (ii) a hybrid *NTTP* × *DynamicShape* axis system that refuses label mismatches at template instantiation when shapes are known and falls back to runtime guards otherwise — one kernel implementation across both paths (Appendix A records a verbatim `static_assert` refusal as the minimum-working-example); and (iii) a *Hexagonal-lite* kernel-backend port carrying three substrates (reference, Eigen, WebGPU/-Dawn) behind one C++20 `concept`. Our axis-bug benchmark suite and a bundle-adjustment case study on the ETH3D dataset operationalise the catch-rate and substrate-cost claims.

1 Introduction

Consider a researcher writing a multi-head attention block. They compose two linear projections, a softmax, and a matmul. The shapes type-check at every step — every intermediate has the right number of dimensions — yet the network silently produces incorrect outputs at training time, because two axes were transposed five lines up (Listing 1):

Listing 1: The motivating bug. The type system sees a (B, H, T, D) tensor where the writer meant (B, T, H, D); the names live only in the comment.

```
// Q : (B, H, T, D)  -- assumed batch, head, time, dim.
auto scores = Q.matmul(K.transpose(-2, -1)) / std::sqrt(D);
auto attn   = softmax(scores, /*axis=*/ -1);
// ...but Q was actually built (B, T, H, D) upstream. Shapes still
// agree, scores still rank-4, attn still numerically finite ---
// gradients silently flow through the wrong heads.
```

This failure mode is endemic. PyTorch's named-tensor prototype was introduced in 2019 specifically to refute it; six years on it remains labelled "prototype, subject to change" and the community-tracking issue records that development has stalled [17]. JAX's analogous `xmap` was deleted in 2024 [11]. The bug is well-known; what is missing is a design that the host language's type system can refuse, not merely a runtime that throws. Existing libraries respond along two axes:

- **String-based runtime notation** (`einops` [19], `einx` [6]): expressive (Einstein-style formulas are the source of truth) but defers all checking to runtime and forfeits the host language's type system.
- **Language-level named tensors with runtime checks** (`Haliax` [22], `Penzai` [8], `xarray` [10], deprecated `torch.named_tensor` [17]): axis names are first-class values, but verification still happens at runtime.

The thesis. A named-axis library should be able to refute the attention-block bug *at compile time, when the shapes are known*, while still permitting the runtime-shape case that batch processing demands. The test of a design is not whether axis names are present but whether they participate in the type system.

Contributions. This paper presents the `tensor` library, organised around three design moves:

1. **The `_tex` LaTeX bridge** (Section 3): a user-defined literal that parses a LaTeX-math subset (`R"(c_{ij} = a_i b_j)"_tex`) into a named-axis `Expression` AST. The AST's index structure — the names attached to each `IndexedVar` and the bound index on each `Sum` — is what the type system reasons about, so the same formula composes with the static-axis machinery of contribution (ii) at no boundary cost.
2. **Hybrid axis system** (Section 4): non-type template parameters carry compile-time axis sizes; `DynamicShape` carries the runtime-determined cases; the two compose without bifurcating the kernel API.

3. **Hexagonal-lite kernel-backend port** (Section 5): a single port boundary admits three substrates (reference, Eigen, WebGPU-via-Dawn [7]) without leaking substrate concerns into the named-axis layer.

Roadmap. Section 2 surveys the axis-bug taxonomy and the prior-art landscape. Sections 3 to 5 present the three design moves. Section 6 reports the static-catch benchmark and a bundle-adjustment case study across three backends. Section 7 positions against contemporaneous work. Section 8 discusses limitations.

2 Background & Motivation

2.1 The Axis-Bug Taxonomy

We catalogue five recurring axis-bug classes (Table 1). Each class is shape-compatible — the offending program type-checks, runs, and silently produces incorrect output — so a position-typed shape system cannot refute any of them.

2.2 The Landscape

Existing approaches to axis naming differ on two orthogonal axes: *where the axis name lives* and *when it is checked* (Table 2). The space splits into three clusters:

String-based runtime. `einops` [19] and `einx` [6] encode axis intent in formula strings interpreted on each call. The notation is expressive (Einstein-style patterns drive contractions, reshapes, and reductions from one source-of-truth string) but every check fires at runtime and nothing about the formula reaches the host language's type system.

Named-value runtime. `Haliax` [22], `Penzai` [8], `xarray` [10], and the prototype `torch.named_tensor` [17] carry axis names as runtime values attached to each tensor (a `NamedArray` field, an `xarray.DataArray` dimension attribute, the `names=` kwarg). Axis mismatch is caught at the first operation that touches both tensors — the failure mode is no longer "silent wrong output" but "crash at runtime," which is strictly better, yet still misses the opportunity to refuse the program at compile time when the shapes are known. The PyTorch implementation has remained labelled "prototype" since its 2019 introduction and development has stalled [17]; JAX's analogous `xmap` was deleted in 2024 in favour of `shard_map` [11].

Type-level (rare). Outside of compiler-IR frameworks (TACO, MLIR `linalg`), only small static-C++ entrants attempt type-level axis names: Einsums encodes index packs as templated literals, Fastor encodes positions as template parameters. Neither admits the runtime-shape case without escape-hatching back to untyped views.

2.3 What remains open

The landscape leaves an explicit gap: *compile-time axis checking that does not abandon the runtime-shape case*. The static approaches in the literature require either fully static shapes (no batch flexibility) or full type erasure at the kernel boundary. Our design target is to recover the latter without giving up the former.

3 Design 1: The `_tex` LaTeX Bridge

3.1 Interface

The user writes the formula the way the corresponding paper would, in a C++ raw-string literal suffixed `_tex`, and feeds it through an Evaluator bound to named tensors:

```
// Bind names -> tensors, then evaluate the formula.
tex::Evaluator<double> ev;
ev.bind("a", DynamicTensor{Shape{Axis{"i", 5}}, /*data*/});
ev.bind("b", DynamicTensor{Shape{Axis{"j", 5}}, /*data*/});

auto c = ev.evaluate(R"(c_{ij} = a_i b_j)"_tex);
// c has named axes (i, j); c[i,j] == a[i] * b[j].
```

The literal `R"(c_{ij} = a_i b_j)"_tex` parses on construction into an `Expression` AST. The `tex::Evaluator` walks the AST against name-to-tensor bindings, asserting that each `IndexedVar`'s subscript list matches its bound tensor's axis labels in order. For example, `R"(a_{ji})"_tex` bound against a tensor whose axes are declared (i, j) is rejected by the Evaluator's subscript-order check.

3.2 Grammar

The accepted grammar is the LaTeX-math subset useful for naming and combining axis-indexed quantities (informal BNF in priority order):

```
equation    := expression ('=' expression)?
expression  := term (('+' | '-') term)*
term        := factor (('*' | '\cdot' | juxt) factor)*
factor      := ('-' factor) | unary
unary       := atom (('/' atom)*)
atom        := '(' expression ')'
             | '\sum' '_' subscript group
             | name subscript?
subscript   := '_' (letter | '{' letters '}')
group       := '{' expression '}' | atom
```

Whitespace is ignored; juxtaposition is implicit multiplication. The parser is a hand-written recursive descent; the grammar is small enough that the implementation fits in a single header (`include/tensor/tex/parser.hpp`).

The choice of LaTeX over an einops-style mini-language is deliberate. The subset that matters — subscripted variables, Ein-

Class	One-line reproduction	What is silently wrong
T1. Transposed contraction	`attn = Q @ K.transpose(-2,-1)` where `Q` is laid out `(B,T,H,D)` rather than the expected `(B,H,T,D)`	Contraction sums over the head axis, not over the model dim. Scores have the right shape, wrong meaning.
T2. Silent broadcast	`x + bias.T` where `x` is `(32,128)` and `bias` is `(1,128)`	The transposed bias `(128,1)` stretches against the batch axis instead of the feature axis; every row gets a different bias scalar.
T3. Reduction over wrong axis	`loss = -(targets * x.log()).sum(dim=0)` on logits `x: (B, C)`	Reduces over the batch dimension; the resulting loss vector has length C instead of being a scalar. Loss curve looks "surprisingly small" and training silently diverges.
T4. Layout swap	A module documented as returning `(H, B, D)` is consumed by code expecting `(B, H, D)` and a `.view(B, H, D)` is applied without a `.permute(1,0,2)` first	Adjacent batch entries get mixed across heads; gradients flow incorrectly. The most common production failure mode (PyTorch named tensors [17] were originally introduced to address exactly this).
T5. Stack vs. concat	`torch.cat([a, b], dim=0)` where the intent was to add a new length-2 axis	Produces a tensor of rank r rather than rank $r+1$; downstream `.view` calls then either silently misalign or crash several operators later, far from the original bug site.

Table 1: Axis-bug taxonomy. All five classes are shape-compatible: no positional shape system rejects them. Section 6.1 benchmarks each library's static / runtime / silent rate against an expanded version of this taxonomy.

stein sums (`R"(\sum_i {a_i b_i})"_tex`), and equations that name a result — is the notation tensor-program papers and textbooks already use. The literal is also *readable as LaTeX*, so the same text that drives evaluation can be re-emitted into the program's typeset output: a Jupyter cell prints back the original formula, closing the loop from notation to running code to rendered documentation.

3.3 Composability with the Static-Axis System

The system-level claim of this section is not about *when* the parse fires — it fires at literal construction today, which is runtime — but about *what shape* it produces. The parsed AST's indices are the same structural objects the static-axis machinery of Section 4 reasons about. The Evaluator's binding step is a type-level operation when the bound tensor's axes are NTTP-encoded, and a runtime check when they are carried in a `DynamicShape`. The Evaluator does not branch on which world it is in; one code path services both.

Concretely, the same literal `R"(c_{ij} = a_i b_j)"_tex` drives three different compile-time / runtime mixes without changing form:

```
// (a) Both operands NTTP-labelled: label match is type-level.
TypedTensor<double, "i"> a_t{{5}, /*data*/};
TypedTensor<double, "j"> b_t{{5}, /*data*/};

// (b) One NTTP, one DynamicShape: name checked statically,
//     size at bind() time.
DynamicTensor<double> b_d{Shape{Axis{"j", k}}, /*data*/};

// (c) Both DynamicShape: both checks at bind() / evaluate().
DynamicTensor<double> a_d{Shape{Axis{"i", n}}, /*data*/};
DynamicTensor<double> b_d2{Shape{Axis{"j", m}}, /*data*/};

tex::Evaluator<double> ev;
ev.bind("a", /* a_t.to_dynamic() or a_d */);
ev.bind("b", /* b_t.to_dynamic() or b_d / b_d2 */);
auto c = ev.evaluate(R"(c_{ij} = a_i b_j)"_tex);
```

The literal and Evaluator are identical across (a)–(c); what changes is only *when* each label and each size is verified — at template instantiation, at `bind()`, or at `evaluate()`. The hybrid system pays for the strongest verification available given how much was known at each construction site, and no more.

3.4 Scope and the Scheduled `consteval` Lift

The parser is presently runtime, executed at the literal's construction site. The natural `consteval` formulation is blocked by C++20's prohibition on `std::unique_ptr` escaping a constant-evaluated context (the current AST is a `unique_ptr`-of-variant tree). A scheduled refactor replaces the `unique_ptr` representation with a fixed-size arena-of-variants whose lifetime is the literal's; that representation does escape constant evaluation and lifts the entire parse into the compiler.

Three properties of the current design are unchanged by the lift, and are this section's contribution:

1. the parser's output type (`Expression`) is identical in both worlds;

Library	Carrier	Check phase
`einops` [19]	string literal	runtime
`einx` [6]	string literal	runtime
`xarray` [10]	runtime value (attribute)	runtime
`Haliax` [22]	runtime value (`NamedArray`)	JAX trace-time†
`Penzai` [8]	runtime value (`NamedArray` PyTree)	JAX trace-time†
`torch.named_tensor` [17]	runtime value (`names=`)	runtime (stalled)
`Nx` [15]	first-class (Elixir)	dialyzer-time (partial)
`tinygrad` [9]	none (positional)	—
`tensor` (this work)	NTTP (`LabelTag<"i">`) *or* runtime `Axis`, per tensor type	**compile time when shapes are static; runtime via the Evaluator otherwise**

Table 2: Where axis names live, and when they are checked. †*JAX trace-time* is the shape check that fires at the first `jit`/`pjit` call — meaningfully later in the dev loop than C++ template instantiation but still before runtime proper. The `tensor` library uniquely occupies the template-instantiation-time-when-known quadrant via `LabelTag<"i">` NTTPs, with explicit fallback to runtime via `DynamicShape`.

2. the Evaluator's binding-and-check pass is identical;

3. the parse cost is paid once per literal, not once per call — today at static-initialiser time, after the lift at compile time.

Grammar-wise, the deliberate non-goals are: no ellipsis, no broadcast operators, no rank-polymorphic patterns, no runtime–constructed strings (the literal arrives as `char const*`). Cases the grammar cannot express are carried by the hybrid system of Section 4.

Why C++ specifically? NTTP *class-type* parameters (C++20, P0732) are what makes `LabelTag<"i">` a structural type whose identity is the label itself; Rust's const generics are not yet at this expressiveness (`const N: &'static str` is stable but value-structural NTTPs remain unstable as of edition 2024), Swift's generic parameters do not admit string-literal NTTPs, and Mojo's parameter system is too young to claim stability. C++ is the language that ships the type machinery our design depends on; the AST representation and parser are otherwise portable in principle. We do not claim the design must remain C++-bound forever, only that it is built where it can be built today.

4 Design 2: Hybrid NTTP × Dynamic-Shape

4.1 The two paths

A named tensor carries its axis information in one of two ways:

Compile-time path axis names *and* sizes are encoded as non-type template parameters. A literal expecting axes (i, j) — e.g. `R"(c_{ij} = a_i b_j)"_tex` — bound through the Evaluator against a tensor whose NTTP axes are (i, k) is refuted at template instantiation; the diagnostic names the offending axis.

Runtime path axis names are encoded as template parameters (compile-time) but sizes live in a `DynamicShape` value (runtime). The compiler refutes axis-*name* mismatches; size mismatches surface as a `ShapeError` at the kernel call site.

4.2 Promotion and Demotion

The two paths are explicitly bridged by one demotion entry point (`TypedTensor::to_dynamic()`) and no implicit promotions.

Demotion (CT → RT) is total and free. A `TypedTensor<T, "i"_ax, "j"_ax>` carries its labels as `FixedString` NTTPs and its extents as a static-rank `Shape<2>`. Its `to_dynamic()` method copies the underlying buffer into a `DynamicTensor<T>` whose `DynamicShape` is populated by reading the NTTP labels out at compile time and the extents at runtime — the names *survive the demotion as runtime strings*. The conversion is total: every TypedTensor demotes; nothing is lost except the type-level guarantee.

```
TypedTensor<double, "i"_ax, "j"_ax> t{{2,
    3}, /*values*/};
DynamicTensor<double> d = t.to_dynamic();
// d.shape() now has Axis{"i", 2}, Axis{"
    j", 3}.
// The type information is erased; the *
    name* information is not.
```

This is what lets a generic Evaluator-driven kernel (Section 3) accept TypedTensor inputs without requiring a templated overload per static-axis pack: the kernel sees one `DynamicTensor` type and the name check uses runtime strings, which the demoted tensor still carries.

Promotion (RT → CT) is opt-in and checked. Going the other direction is not type-safe by construction — the labels read from a `DynamicShape` are runtime data — so the library does not provide an implicit conversion. Instead, the user asserts the expected static labels and the runtime data is verified once:

```
DynamicTensor<double> d = load_from_disk
    (...);
```

```
// d.shape() carries runtime Axis labels and extents.

// Promotion asserts the expected NTTP pack; if the labels do not
// match (or rank differs), construction throws std::invalid_argument
// at the assertion site rather than silently producing a wrong type.
auto t = TypedTensor<double, "i"_ax, "j"_ax>::from_dynamic(d);
```

This is the only point in the system where the runtime label string is read into a type-level decision. The choice to make promotion explicit means the type system retains a useful invariant: a `TypedTensor<T, L...>` value's labels are always `L...` — always known at compile time, never doubted at runtime.

The two-path rule. A kernel is written exactly once, against the runtime path. Static callers reach it via `to_dynamic()`; dynamic callers reach it directly. The compiler-enforced `static_assert(SameLabels<A,B>)` guard in the `TypedTensor` operators (`operator+`, `operator-`, etc.) refuses static-side mismatches before they reach the demotion call; the Evaluator's name-and-rank checks refuse the same mismatches on the dynamic side. The two checks share the intent ("the labels do not match") and differ only in when they fire.

4.3 Kernel boundary

The contract of a kernel is a function template parameterised by the axis structure (compile-time) accepting `DynamicShape` runtime sizes when needed. *The same kernel* services both paths; the runtime-path overhead is one bounds check per call, not a separate kernel implementation.

5 Design 3: Hexagonal-lite Kernel Backend

The named-axis layer should not know whether contractions are computed by hand-rolled loops, an Eigen template instantiation, or a WGSL kernel dispatched to a GPU. *Hexagonal-lite* is a constrained form of Cockburn's Hexagonal architecture [3]: a single port (`KernelBackend`) with three concrete adapters.

5.1 The Port

The port is a C++20 `concept` declared in `include/tensor/core/concepts.hpp`. A substrate that satisfies `KernelBackend` can be plugged into the Domain without further adaptation work:

```
template <class B>
concept KernelBackend = requires(B b) {
    typename B::backend_tag;
} && requires(B b,
              DynamicTensor<double> const& a,
              DynamicTensor<double> const& a2,
              BroadcastPlan const& bplan,
              ContractPlan const& cplan,
              std::vector<std::size_t> const& source_map,
              DynamicShape const& source_shape) {
    // Element-wise binary, same shape.
    { b.add(a, a2) } -> std::same_as<DynamicTensor<double>>;
    { b.sub(a, a2) } -> std::same_as<DynamicTensor<double>>;
    { b.mul(a, a2) } -> std::same_as<DynamicTensor<double>>;
    { b.div(a, a2) } -> std::same_as<DynamicTensor<double>>;
    // Element-wise unary.
    { b.exp(a)  } -> std::same_as<DynamicTensor<double>>;
    { b.log(a)  } -> std::same_as<DynamicTensor<double>>;
    { b.relu(a) } -> std::same_as<DynamicTensor<double>>;
    { b.neg(a)  } -> std::same_as<DynamicTensor<double>>;
    // Broadcast element-wise.
    { b.broadcast_add(a, a2, bplan) } -> std::same_as<DynamicTensor<double>>;
    { b.broadcast_sub(a, a2, bplan) } -> std::same_as<DynamicTensor<double>>;
    { b.broadcast_mul(a, a2, bplan) } -> std::same_as<DynamicTensor<double>>;
    // Contraction (Einstein-style).
    { b.contract(a, a2, cplan) } -> std::same_as<DynamicTensor<double>>;
    // Reduction.
    { b.reduce_sum(a) } -> std::same_as<double>;
    // Unbroadcast: used by autograd to reduce dL/dout to an input's shape.
    { b.unbroadcast(a, source_map, source_shape) }
        -> std::same_as<DynamicTensor<double>>;
};
```

Three deliberate choices shape this surface:

1. **The Domain pre-computes the plans.** `BroadcastPlan` and `ContractPlan` are computed once in `tensor::core` and handed to the backend, so a substrate never re-implements axis matching or reduction-set inference. Substrates differ on *how* to evaluate a plan, not on *whether* a plan is well-formed.

2. **Unbroadcast is in the port.** The reverse of a broadcasted

op — reducing `dL/dout` back to an input's shape — belongs to the substrate because Eigen and Kokkos can implement it as a single fused reduction. Keeping it outside autograd lets the autograd layer compose port methods without per-substrate special cases.

3. **Per-op backward closures are *not* in the port.** They live in `tensor::autograd` and compose port methods. The substrate owns "how to compute," the autograd layer owns "how to differentiate."

Each concrete substrate ends with a `static_assert(KernelBackend<Backend>)`, so any divergence between the port surface and an adapter is a compile error at the point where the adapter is defined, not a runtime surprise.

5.2 Substrates

Reference portable naive nested-loop implementation; defines the semantics that the other substrates must match.

Eigen BLAS-routed contractions for CPU performance baseline; loop fusion via Eigen's expression templates.

WebGPU via Dawn [7] GPU substrate compiled to WGSL; vcpkg-vendored Dawn to keep substrate provisioning reproducible in CI.

The choice of substrate is a CMake variable (`-DTENSOR_KERNEL_BACKEND={reference,eigen,webgpu}`) — the named-axis layer is unchanged across all three.

6 Evaluation

We evaluate two distinct questions: *does the static-axis path actually refuse the axis-bug classes of Section 2.1?* (Section 6.1), and *does the Hexagonal-lite substrate boundary leak performance through its abstraction?* (Section 6.2). The static-catch numbers are the paper's distinctive evaluation; the runtime case study sanity-checks that substrate substitutability does not cost an order of magnitude.

Status of results. Methodology, harness scaffolding, and the comparison protocol are final. Numerical results are collected by a dedicated benchmark harness (`bench/bench_mlsys_paper_case_study.py`), implementation of which is scheduled for September 2026, ahead of the conference deadline; the present submission carries the fully-specified protocols (this section) and an empty result table that the harness output drops into directly. Where numbers below read **[BENCH: ·]**, the design of the measurement is locked but the number itself is pending the harness landing.

6.1 Static-catch Benchmark

Suite construction. The benchmark suite has $5 \times 10 = 50$ *bug instances* (ten per axis-bug class T1–T5 of Table 1) plus 25 *control programs* that exercise the same operators *correctly*. The controls let us also report a per-library *false-positive rate*: a system that refuses everything would score 100% on bugs but would be useless on the controls. Each entry is a self-contained one-screen example. Entries derive from the documented community reports in Table 1 rather than from inspection of any single library's source.

Per-library translations. Each bug instance and each control is implemented in five idiomatic target forms: `einops`, `einx`, `Haliax`, `Penzai`, and `tensor` (the latter once via the static `TypedTensor` path and once via the dynamic `DynamicTensor` + `_tex` path, scored separately). Translations are written by one author and reviewed by a second author against the target library's official documentation; the per-entry annotation links to the documentation page that motivated each idiomatic choice. The total artefact is 75 entries $\times$ 5 libraries + 75 $\times$ 1 (the static `tensor` second variant) $=$ 450 translated programs, all shipped in `bench/static_catch_suite/`.

Per-entry scoring. The harness compiles or invokes each program in a freshly-instantiated target environment and assigns one of four outcomes:

CT (compile / trace time) the program fails before any tensor is allocated. For statically-typed libraries (`tensor`-static): a `static_assert` or template-instantiation error. For JAX-backed libraries: a `ShapeError`/`TracerArrayConversionError` at the first `jit`/`pjit` call.

RT (runtime, first call) the program reaches the offending operator and raises (`ShapeError`, `TypeError`, `ValueError`, or library-specific).

Silent the program runs to completion. The result is then compared to the entry's recorded *intended* output (the output a correct implementation of the programmer's stated goal would produce). For *structural* bugs (T5 stack/concat) the comparison is on shape; for *numerical* bugs (T1–T4) the comparison is on ℓ_∞ difference against the recorded reference at tolerance 10^{-6}. Anything not flagged at CT or RT and not matching the reference is **Silent**.

Pass (control programs only) the program runs and matches the reference. Bug instances that score **Pass** indicate an instrumentation defect and are excluded with a suite-level retraction note.

Headline metric. The per-library catch rate over the 50 bug instances, with the false-positive rate over the 25 controls reported alongside:

The `tensor` library appears on *two* rows because its static and dynamic paths offer materially different guarantees; we report both so the cost of opting into the static path (slightly less ergonomic, more boilerplate) is visible against its benefit (higher CT column).

Library	CT %	RT %	Silent %	FP %
`einops` [19]	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]
`einx` [6]	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]
`Haliax` [22]	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]
`Penzai` [8]	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]
`tensor` (ours, static)	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]
`tensor` (ours, dynamic)	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]

Table 3: Static-catch rate per library across the 50-bug suite, with the false-positive rate (FP) measured against the 25-control set. *CT+RT* (sum of the first two columns) is the per-library safety score; *Silent* is the residual silent-wrong-output rate. Hypothesis: the static `tensor` row's CT column dominates all other rows, the dynamic row matches the named-value-runtime cluster, and FP is at noise for every row. Numbers pending the harness implementation (see *Status of results* above).

6.2 Bundle-adjustment Case Study

Workload. We assemble residuals and Jacobians for the standard bundle-adjustment (BA) nonlinear least-squares problem on the *courtyard* scene of the ETH3D multi-view stereo benchmark [21] (38 high-resolution images, COLMAP-format poses obtained via the canonical incremental SfM pipeline [20]), with a 50-view $\times$ 200-landmark subselection chosen so the problem fits in GPU memory on a consumer card while remaining large enough to exercise the substrates' scaling. A second pass on the *fountain-P11* scene of the Strecha benchmark [23] (11 high-resolution images with LIDAR ground-truth) supplies a smaller, classic validation scene; the BAL dataset [1] contributes a *Ladybug-49* small-scale ablation. The named-axis structure (`view`, `landmark`, `pose-dof`, `obs`) is invariant across the implementations being compared; what varies is the library used to express the Jacobian-assembly contractions and the substrate that evaluates them.

The case-study code is the same code as `tutorials/09_bundle-adjustment.ipynb` of the source distribution, scaled up: the tutorial introduces the formulation on a synthetic 5-view $\times$ 20-landmark mini-scene; the bench harness swaps the data source for the ETH3D / Strecha / Ladybug-49 scenes and instruments the timings reported below.

Backends compared. Three substrates of our system, plus two cross-library baselines:

- *`tensor` / reference* — the portable nested-loop reference backend (Section 5.1).
- *`tensor` / Eigen* — the BLAS-routed CPU backend.
- *`tensor` / WebGPU (Dawn)* — the GPU substrate.
- *`einops`* — baseline contracting via NumPy.
- *`Haliax` on JAX* — baseline named-tensor library on the JAX backend.

Backend	t_r (ms)	t_g (ms)	peak (MB)
`tensor` / reference	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]
`tensor` / Eigen	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]
`tensor` / WebGPU (Dawn)	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]
`einops` / NumPy	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]
`Haliax` / JAX	[BENCH: ·]	[BENCH: ·]	[BENCH: ·]

Table 4: Bundle-adjustment case study results across three of our substrates and two cross-library baselines. t_r = time to residual, t_g = time to gradient. Hypothesis: the three `tensor` rows differ by substrate cost only — the named-axis layer adds no measurable overhead. Numbers pending harness landing.

Residual. For view v observing landmark l the reprojection residual is

$$\mathbf{r}_{vl} = \pi\big(R_v \mathbf{p}_l + \mathbf{t}_v\big) - \mathbf{o}_{vl},$$

with $\pi(x, y, z) = (x/z,\ \ y/z)$ the canonical pinhole projection under normalised intrinsics ($f = 1$, principal point at origin) and $\mathbf{o}_{vl} \in \mathbb{R}^2$ the observed image coordinate. The named-axis structure of the residual tensor is $(\mathsf{view}, \mathsf{landmark}, \mathsf{uv})$; the same formula expressed in `_tex` is what `tutorials/09_bundle-adjustment.ipynb` shows on a 1-D simplification. Initial poses come from COLMAP [20]; we perturb each R_v by a random rotation of magnitude $0.5°$ and each $\mathbf{t}_v$ by Gaussian noise of standard deviation $\sigma = 0.01$ (units of the scene extent) so that the optimisation begins off-equilibrium.

Metrics and statistical protocol. Three measurements per (backend, scene) pair:

t_r wall-clock time for one forward pass producing the $50 \cdot 200 \cdot 2 = 20{,}000$-entry residual.

t_g wall-clock time for one autograd-derived gradient pass.

peak peak resident-set memory during $t_r + t_g$.

We report the *median* over n=30 runs after n=5 warm-up runs and the 95% bootstrap confidence interval. Median + IQR are preferred over mean + s.d. because wall-clock distributions on shared machines are right-skewed by occasional pre-emption. Before timing, every backend pair is required to produce numerically identical residuals on the test inputs (ℓ_∞ difference $< 10^{-9}$, FP64 throughout); a backend whose output diverges disqualifies its row from the timing report.

Hardware. The CPU substrates (reference, Eigen, einops, Haliax) run on a GitHub-hosted ubuntu-22.04 runner (4 vCPU, 16 GB RAM, no GPU). The WebGPU lane runs on the maintainer's developer machine (**NVIDIA GeForce RTX 4070 Mobile**, 8 GB VRAM, Dawn r6814 with the Vulkan backend, Ubuntu 24.04, driver 550.142). The shared-runner numbers are reproducible by anyone with a free GitHub account; the WebGPU numbers should be read as the *tensor*-internal cross-substrate *ratio* rather than as an absolute claim against datacentre hardware.

A scaling plot of time-to-gradient versus view count ($N_v \in \{10, 25, 50, 100, 200\}$) accompanies the table if page budget allows.

6.3 Threats to Validity

Suite size and curation bias. A $\sim$50-entry static-catch suite is small. We mitigate single-author bias by deriving entries directly from the bug taxonomy (Table 1, each class drawn from documented community reports), not from inspection of any particular library's source. The suite, harness, and per-library idiomatic reproductions ship in the `bench/` directory of the supplementary material so a reviewer can add a class or a library and re-score.

Idiomatic fairness. Each per-library entry is written by someone familiar with the target library's idioms. Mis-classifying a fair use as "Silent" because the writer did not reach for the library's correct guard would inflate our advantage; the harness therefore records, for each entry, a one-line annotation linking to the relevant guidance in the target library's documentation. Where we cannot find such guidance, the entry is annotated "no guard documented" rather than scored.

Hardware variability for the case study. The WebGPU lane runs on a single consumer GPU; absolute numbers are not portable to data-centre hardware. We report the `tensor`-internal cross-substrate ratios as the primary signal and use absolute baseline numbers only to anchor units.

Construct validity of the BA case study. Bundle adjustment exercises high-rank index reasoning (`view × landmark`, sparse 2D residual) but under-exercises broadcasting (where named-axis libraries diverge most). The three-scene rollout (ETH3D *courtyard* primary, Strecha *fountain-P11* validation, BAL *Ladybug-49* small-scale ablation) covers a span of camera/landmark ratios but remains within the BA class. A second case study sensitive to broadcasting (e.g. multi-head attention forward) would strengthen external validity; we record this as deliberately out of scope for the present 10-page submission and note it in Section 8.

Reproducibility package. The release ships `bench/` containing: (a) the 75 static-catch programs across five libraries (450 translated files); (b) a downloader script for ETH3D + Strecha + BAL scenes; (c) the harness driver `bench_mlsys_paper_case_study.py`; (d) the resulting `results.json` schema (one record per scoring outcome or timing measurement); (e) the plotting scripts that render Table 3 and Table 4 from `results.json`; (f) a pinned conda environment file and a `Dockerfile` for the WebGPU lane. End-to-end re-runs on a fresh runner reproduce the reported numbers to within the published confidence intervals.

7 Related Work

We position against four clusters: string-encoded notation libraries, runtime-named-value libraries, type-level static-axis libraries, and the underlying notation literature. Table 2 placed each on the (carrier, check-phase) grid; this section adds the qualitative comparison that the grid compresses.

String-encoded notation. `einops` [19] and `einx` [6] pioneered the ergonomic shift from positional to formula-driven tensor manipulation: an `einops.rearrange("b h t d -> b (t h) d", x)` replaces three `.transpose`/`.view` calls with one declarative pattern. `einx` generalises the formula language to a richer set of patterns including stacked broadcasts. Both libraries parse the formula string on every call and reach the host language's type system only as opaque `Tensor`-returning functions; mismatches surface as exceptions, not as compile-time errors. The `tensor` library carries over the design instinct (a literal formula is the right authoring surface) but disagrees on the parse phase: our `_tex` literal exposes its parsed structure to the type system so the static-axis path of Section 4 can act on it. We adopt a LaTeX-math subset rather than einops's bespoke grammar so the same literal can be re-rendered as typeset documentation.

Runtime named values. `Haliax` [22] (Stanford CRFM, JAX-backed, powering 70B-parameter LLM training on TPU v4-2048), `Penzai` [8] (Google DeepMind, JAX-backed), `xarray` [10] (scientific computing, NumPy/Dask-backed), `Nx` [15] (Elixir, EXLA-bridged), and the prototype `torch.named_tensor` [17] all promote axis labels from comments to values. The check phase varies within this cluster: `Haliax` and `Penzai` benefit from JAX's trace-time shape verification at the first `jit`/`pjit` call (better than pure runtime, but still after C++-style template instantiation in the dev cycle); `xarray` and `torch` are fully runtime. This converts T1–T5 of Table 1 from "silent wrong output" to "runtime exception" — a strict improvement. Two structural limits remain: (i) when the tensor's shape *is* statically known, the runtime or trace-time check is doing avoidable work and emitting an avoidable diagnostic; (ii) the named-tensor lineage has had difficulty stabilising ergonomically (PyTorch's stalled prototype; JAX's deletion of `xmap` [11]), suggesting that runtime-only is not the final design point. Our hybrid system uses the runtime-named approach where shapes genuinely are dynamic and reaches for static-axis types when they are not.

Type-level static axes. Outside compiler frameworks, a small C++ literature attempts type-level axis tracking: `Einsums` [5] encodes Einstein-pattern packs as template arguments; `Fastor` [16] encodes positional axis extents as template parameters and the `einsum` as a formula at the type level; `Tenseur` [13] explores lazy expression templates over BLAS. These libraries reject mismatches at compile time when the shape is fully static, but offer no escape

hatch to a runtime-shape regime — a tensor whose rank or extent is learned from disk cannot be expressed without dropping out of the typed API entirely. Our hybrid system shares the static rigour of this cluster on the static path and adds the missing runtime path via `DynamicTensor` + `TypedTensor::from_dynamic` (Section 4.2).

Compiler-IR and DSLs. `TACO` [12], `COMET` [4], and MLIR's `linalg`/`tensor` dialects [14] express index-name semantics formally, but at the compiler-IR level: the user writes either a target-DSL string or invokes the compiler toolchain rather than calling library-level operators. `Halide` [18] similarly separates algorithm from schedule. `tinygrad` [9] keeps the API at twelve primitive ops and infers shapes positionally. These are useful reference points for what "maximal axis discipline" looks like at the codegen level; our work targets the library-API level with no compiler stage.

Notation and formalism. Chiang and Rush [2] formalise named-tensor notation as a typed calculus and present a paper notation usable across linear algebra textbooks and ML papers. We adopt their notation as the authoring surface (Section 3.2 accepts a restricted LaTeX subset compatible with theirs) and supply the type-system implementation that the calculus invites but which their paper does not provide: an in-language host (C++20) where the named-axis structure is what the typechecker reasons about.

8 Discussion and Limitations

When to reach for which path. The hybrid system offers three settings and an upgrade door; the rule of thumb is to take the strongest guarantee that the calling context supports:

- *All shapes and labels known at the call site* (e.g. a fixed model architecture, a layer's weight tensor at module construction): use `TypedTensor<T, L...>`. Label mismatches are refuted at template instantiation; the compiler diagnostic names the offending axis.
- *Labels known statically, sizes vary at runtime* (e.g. the batch dimension): use `TypedTensor` for the labels and let extents be runtime — the static check refuses wrong-name compositions; the runtime check fires on wrong-size compositions. This is the most common case in ML kernels.
- *Labels and sizes both runtime* (e.g. tensors deserialised from disk, user-supplied inputs at inference time): use `DynamicTensor` directly. Both checks fire at the first operator call; the failure mode is a `ShapeError`, not undefined behaviour.
- *Bootstrap into the static path*: when a tensor enters the system as `DynamicTensor` but downstream code benefits from static guarantees, an explicit `TypedTensor::from_dynamic` call verifies the runtime labels against the expected NTTP pack once and crosses the boundary (Section 4.2). After that point all downstream operations use the static path.

Limitations. **(L1) UDL grammar is deliberately small.** The `_tex` grammar (Section 3.2) accepts subscripted variables, sums, equations, and the usual arithmetic operators. It does not accept rank-polymorphic patterns (einops's `...` ellipsis), no broadcast-aware patterns (einx-style `(t h)`-grouping), and no runtime-constructed formula strings. The hybrid system absorbs the runtime-shape case (Section 4), but the design does not currently express rank-polymorphism as a first-class feature.

(L2) Diagnostic quality on the static path is template-deep. A label mismatch in `TypedTensor<T, "i"_ax> + TypedTensor<T, "j"_ax>` produces a `static_assert` failure with the intended message ("axis labels must match"), but it is wrapped in the operator's template instantiation backtrace. We provide a `assert_same_label` helper that surfaces the relevant labels directly, and the backtrace begins at the operator site rather than deep in the compiler — yet C++'s default error rendering is far from the ergonomic that languages like Rust or Swift have established as the bar. Diagnostic surface is genuine future work.

(L3) WebGPU substrate portability is constrained. The Hexagonal-lite `webgpu` adapter is built atop Dawn [7] and inherits Dawn's platform matrix (Windows, macOS, Linux, ChromiumOS; Android is preview). On Linux, software emulation paths exist but performance numbers should not be extrapolated across them. Substrates beyond reference / Eigen / WebGPU — Kokkos, SYCL, or direct CUDA — are not yet implemented; the port surface (Section 5.1) is designed to receive them without changes.

(L4) Single-author empirical coverage. The static-catch suite and the BA case study were curated by the authors and may reflect blind spots that an external reviewer would catch. The harness in `bench/` is structured to accept contributed entries; the per-entry annotation linking each "Silent" classification to the target library's documentation is intended as a falsification handle for readers.

Future work. The primary scheduled lift is moving the `_tex` parser from runtime (literal-construction time) to `consteval` (compile time) by replacing the `unique_ptr`-of-variant AST with a fixed-size arena representation that escapes constant evaluation (Section 3.4); the output type and Evaluator pass are unchanged, so this is a mechanical refactor whose payoff is moving parse cost from program startup to compile time. Beyond that: extending the grammar towards rank-polymorphic patterns without losing the structural-index property; auto-tuned substrate selection based on shape and target; integration with Triton-class kernel languages at the Hexagonal-lite adapter layer.

9 Conclusion

Named-axis libraries should treat the host language's type system as load-bearing infrastructure, not as a place to attach

metadata. The `tensor` library demonstrates that the static and dynamic worlds compose: a LaTeX-bridging `_tex` UDL whose AST shape is the same object the type system reasons about, a hybrid NTTP × DynamicShape axis system that shares one kernel across both paths, and a Hexagonal-lite kernel backend that demonstrates substrate substitutability across three concrete adapters. The static-axis path is what lets the type system refuse the axis-bug taxonomy of Section 2.1; the runtime path is what keeps the design livable for the cases statically-known shapes do not cover.

A Reproducing the Static-Axis Refusal

The static-axis claim of Section 4 — "`TypedTensor` operators refuse axis-label mismatches at template instantiation time" — is a small enough property to verify in five lines. We record the demo source and the verbatim compiler diagnostic so a reviewer can reproduce locally:

Listing 2: Demo source: two tensors with mismatched axis labels are added.

```
#include <tensor/core/typed_tensor.hpp>
#include <tensor/core/label_tag.hpp>
using namespace tensor::core::literals;

int main() {
    tensor::core::TypedTensor<double, "i
        "> a{{3}, {1, 2, 3}};
    tensor::core::TypedTensor<double, "j
        "> b{{3}, {4, 5, 6}};
    auto c = a + b;   // requested: align
         (i) with (j).
    return 0;
}
```

Listing 3: GCC 11.4 diagnostic, essential lines only.

```
typed_tensor.hpp:128:19: error: static assertion
    failed:
  TypedTensor: axis labels must match at compile time.
  Drop into DynamicTensor with .to_dynamic() if you
       need broadcast.
note: the expression same_labels_impl<A, B>::value
  [with A = TypedTensor<double, {"i"}>;
        B = TypedTensor<double, {"j"}>]
  evaluated to 'false'
```

Two properties of this diagnostic matter for the central claim:

1. the error is a `static_assert` fired at template instantiation time, not at runtime — the program never starts;

2. the diagnostic names the offending axis labels directly (`{"i"}` vs `{"j"}`) and the recommended escape (`.to_dynamic()`), exemplifying the diagnostic-quality work flagged as **L2** in Section 8.

The same refusal extends to the contraction operator (where label matching is partial rather than total) and to higher-rank tensors; the demo above is the minimal reproducer.

References

[1] Sameer Agarwal, Noah Snavely, Steven M. Seitz, and Richard Szeliski. Bundle adjustment in the large. In *European Conference on Computer Vision (ECCV)*, 2010. The BAL observation dataset; `https://grail.cs.washington.edu/projects/bal/`. Used as a small-scale validation supplement to the ETH3D case study.

[2] David Chiang and Alexander M. Rush. Named tensor notation. In *Transactions on Machine Learning Research*, 2024. Formalises named-axis notation as a typed calculus.

[3] Alistair Cockburn. Hexagonal architecture (Ports and Adapters). `https://alistair.cockburn.us/hexagonal-architecture/`, 2005. Originally published 2005; revised through 2024.

[4] COMET contributors (PNNL). COMET: Domain-specific COMpiler for Extreme-scale Tensor algebra. `https://github.com/pnnl/COMET`, 2021–2026.

[5] Einsums contributors. Einsums: A C++ tensor library with compile-time einstein-notation patterns. `https://einsums.github.io/Einsums/`, 2023–2026.

[6] Florian Flick. Einx: Universal tensor operations in Einstein-inspired notation. In *International Conference on Learning Representations (ICLR), oral*, 2026. Project: `https://github.com/fferflo/einx`.

[7] Google Chrome. Dawn: A WebGPU implementation. `https://dawn.googlesource.com/dawn`, 2020–2026.

[8] Google DeepMind and contributors. Penzai: A jax research toolkit with named tensors. `https://github.com/google-deepmind/penzai`, 2024–2026.

[9] George Hotz and tinygrad contributors. tinygrad: A deep-learning framework for the small. `https://github.com/tinygrad/tinygrad`, 2020–2026.

[10] Stephan Hoyer and Joseph Hamman. xarray: N-D labeled arrays and datasets in Python. *Journal of Open Research Software*, 2017.

[11] JAX maintainers. Deprecation and removal of `xmap` from JAX. `https://github.com/jax-ml/jax/discussions/20312`, 2024.

[12] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. The tensor algebra compiler. In *Proc. ACM Program. Lang. (OOPSLA)*, 2017.

[13] Istvan Marc. On designing Tenseur: a C++ tensor library with lazy evaluation. `https://istmarc.github.io/post/2024/10/27/on-designing-tenseur-a-c-tensor-library-with-l` 2024.

[14] MLIR project. MLIR `linalg` Dialect. `https://mlir.llvm.org/docs/Dialects/Linalg/`, 2020–2026.

[15] Nx contributors. Nx: Multi-dimensional arrays (Tensors) and numerical definitions for Elixir. `https://github.com/elixir-nx/nx`, 2021–2026. EXLA backend bridges to XLA via Erlang ports; first-class named axes via Elixir pattern matching.

[16] Roman Poya and Fastor contributors. Fastor: A lightweight high-performance tensor algebra library for modern C++. `https://github.com/romeric/Fastor`, 2017–2026.

[17] PyTorch. Named tensors (experimental). `https://pytorch.org/docs/stable/named_tensor.html`, 2019–2026. API marked experimental since 2019; community-tracking status at `https://github.com/pytorch/pytorch/issues/60832`.

[18] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *PLDI*, 2013.

[19] Alex Rogozhnikov. Einops: Clear and reliable tensor manipulations with Einstein-like notation. In *International Conference on Learning Representations (ICLR)*, 2022. Project: `https://github.com/arogozhnikov/einops`.

[20] Johannes L. Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. The COLMAP pipeline; de-facto reference for modern incremental SfM.

[21] Thomas Schöps, Johannes L. Schönberger, Silvano Galliani, Torsten Sattler, Konrad Schindler, Marc Pollefeys, and Andreas Geiger. A multi-view stereo benchmark with high-resolution images and multi-camera videos. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. The ETH3D benchmark; `https://www.eth3d.net/`. High-resolution images with LIDAR-derived ground truth, COLMAP-format poses.

[22] Stanford CRFM and contributors. Haliax: Named tensors for jax. `https://github.com/stanford-crfm/haliax`, 2023–2026.

[23] Christoph Strecha, Wolfgang von Hansen, Luc Van Gool, Pascal Fua, and Ulrich Thoennessen. On benchmarking camera calibration and multi-view stereo for high resolution imagery. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2008. The classic "Castle / Fountain / Herzjesu" multi-view benchmark; `https://www.epfl.ch/labs/cvlab/data/data-strechamvs/`.